JOUR 04 BIS / TP 02 — V2 · VERSION GUIDÉE (V2)

Resource Detail

Ouvrir un véhicule et une mission par identifiant, et distinguer ressource absente vs 404.

---

*Durée indicative : **40 à 45 min** · Organisation : **individuel** · Évaluation :** non noté*

Support : IDE, à partir du TP 1 (starter `tp-02-resource-detail` si vous changez de dossier)

Contexte

Les routes `/fleet/:vehicleId` et `/missions/:missionId` existent, mais les pages sont des coquilles. Un même modèle de route sert **toutes** les ressources : le segment se lit avec `useParams`.

Ce paramètre est une **entrée externe**. Il peut être absent, mal encodé, ou ne correspondre à aucune donnée. Ce n'est pas un crash, pas un écran blanc, pas une redirection silencieuse.

Deux absences différentes :

- `/fleet/unknown` — la route existe, la ressource non → `ResourceNotFound`
- `/pizza-on-mars` — **aucune** route → `NotFoundPage` (`path="*"`)

Elles peuvent partager un ton (console spatiale), pas le même message.

`Link` sur une carte ouvre la fiche. `useNavigate` sert **après une action** (« Ouvrir le contrôle »), pas pour remplacer tous les liens.

Résultat attendu

1. Chaque carte de flotte mène à `/fleet/<id>`. La fiche montre identité, statut, et au moins une métrique de catégorie.

2. `MissionsPage` liste au moins deux missions. Chaque mission mène à `/missions/<id>` (objectif, fenêtre, véhicule assigné, phase).

3. Lien **Retour** vers la liste (`Link`, pas `navigate(-1)` comme seul moyen).

4. `/fleet/unknown` et `/missions/does-not-exist` : état « introuvable » + lien vers la liste — **pas** la 404 globale.

5. `/pizza-on-mars` : `NotFoundPage` (titre clair, lien vers `/` ou `/control`, focus visible). F5 réaffiche la 404.

6. Un bouton **Ouvrir le contrôle** sur la fiche mission (ou véhicule) navigue vers `/control` ou `/control/<missionId>`.

7. `npx tsc --noEmit` est vert.

Étapes

1. Lire le paramètre véhicule

Dans `VehicleDetailPage` :

```
const { vehicleId } = useParams<{ vehicleId: string }>()
```

Cherchez dans **la même source** que `FleetPage`. Si `vehicleId` est `undefined`, traitez-le comme introuvable.

Sur `VehicleCard`, un `Link` vers `vehiclePath(vehicle.id)` (titre ou « Ouvrir »). Pas `window.location`. `key` de liste = `vehicle.id`.

Quand le véhicule existe : nom, id, statut (`StatusBadge` si vous l'avez), métriques de **sa** catégorie (narrowing sur `kind`). Lien retour vers `FLEET`. Ne recopiez pas tout `VehicleCard`.

2. Ressource absente

Créez `src/components/ResourceNotFound.tsx` :

- titre du type « Véhicule introuvable » (le mot ressource sera un prop) ;
- l'id demandé, s'il existe ;
- un `Link` vers la liste concernée.

Dans la page : `if (!vehicle) return <ResourceNotFound resource="véhicule" id={vehicleId} />`. Pas de `navigate("/fleet")` automatique.

3. Missions : liste + fiche

Type `Mission` (ou celui du starter) : `id`, `name`, `objective`, `launchWindow`, `vehicleId`, `phase`. Au moins deux missions, dont une liée à un véhicule réel.

- `MissionsPage` : `Link` vers `missionPath(id)`.
- `MissionDetailPage` : `useParams`, recherche, cinq champs, lien retour vers `/missions`.
- Véhicule assigné : `Link` vers `vehiclePath(vehicleId)` s'il existe.
- Id inconnu → **le même** `ResourceNotFound` (`resource="mission"`).

Sur la fiche, un bouton **Ouvrir le contrôle** appelle `navigate(...)` **après** le clic. Ce n'est pas le lien retour.

4. 404 utile

Dans `App.tsx`, `path="*"` pointe vers `NotFoundPage`, **après** les autres routes.

Dans `NotFoundPage` :

- un `h1` (« 404 — Signal introuvable ») ;
- un paragraphe : **le module** n'existe pas (pas la ressource) ;
- un `Link` réel (« Retourner au centre de contrôle » ou vers `/`).

Interdit : `href="#"`, `onClick` vide, `window.history.back()` comme seul moyen. Traitement visuel simple (fond, cadre, typo). Contraste lisible. Le lien reste le premier élément tabbable utile.

Ouvrez côte à côte `/fleet/unknown` et `/pizza-on-mars` : les titres doivent différer.

5. Vérifier les URLs hostiles

Sans cliquer, saisissez :

- `/fleet/aquila-1` (ou l'id réel d'un véhicule)
- `/fleet/unknown` et `/fleet/%20`
- `/missions/<id-valide>` et `/missions/does-not-exist`
- `/pizza-on-mars`
- F5 sur une fiche valide et sur la 404

```
npx tsc --noEmit
```

Livrables

- `VehicleDetailPage` + `MissionDetailPage` fonctionnelles
- `ResourceNotFound` + `NotFoundPage` distinctes
- `Link` depuis les cartes
- type-check sans erreur
- commit :

```
git add src
git commit -m "feat(session-04): resource detail and lost-in-space 404"
```

Suite — TP 3 Establish Ground Link

Les routes sont ouvertes, les données sont encore **locales**. Avant de brancher le réseau, rappelez-vous : un `useEffect` n'est pas un tiroir à logique. Le TP 3 pose des **services** `getVehicles` / `getMissions`, un vrai `fetch` avec dépendances stables et `AbortController`, et la conversion DTO → modèle UI.